Programming Assignment 4
Cosmic Concordance

COP3503 Computer Science 2 Spring 2026

Due Date

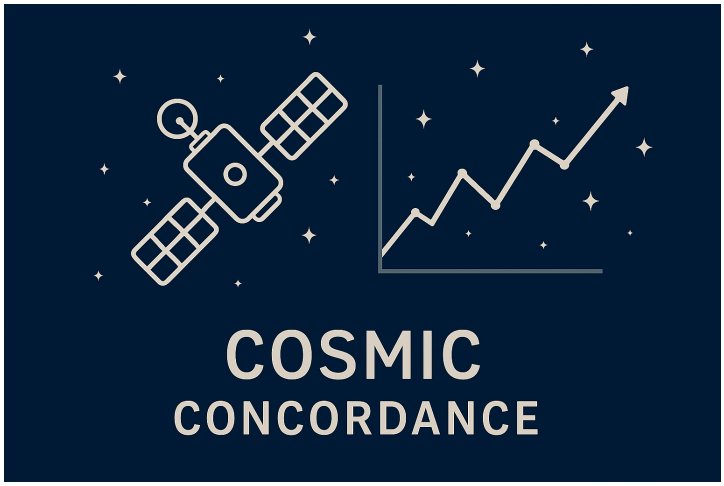
The assignment is due on **Sunday, April 05 at 11:59 PM EST** via Webcourses. **Do not email the professor or TAs. Submissions by email will not be accepted.**

Assignments may be submitted up to 24 hours late with a penalty. Please refer to the syllabus for details. To receive full credit, submit on time and allow extra time for uploading. Last minute submissions often fail due to slow Internet, resulting in late penalties.

The **Webcourses timestamp** determines submission time. If your submission is even one second past the deadline, it will be marked late. Internet issues are not an excuse. Please plan accordingly.

Assignment Description

Two space satellites, Astra-1 and Astra-2, have been monitoring cosmic radiation streams for a while. Each satellite records readings as a sequence of integers over time. Scientists noticed that certain radiation patterns show gradual upward trends that correspond to rare cosmic events. When both satellite observatories witness the same upward-moving signal, scientists gain high confidence that a real cosmic event occurred. Incidentally, you are part of this group of scientists.



The universe, however, is noisy. Readings can briefly dip due to solar wind, instrument jitter, or cosmic dust. The scientists tolerate a limited number of such dips. Also, the instruments are not perfectly identical, so two readings are considered the same if they are numerically close (within a tolerance).

As an expert member of this scientist group, your task is to recover the longest shared radiation signature that:

- appears in both logs in order (you may skip readings),
- mostly rises by at least G at each step,
- allows at most K noise dips (where a step that fails the growth requirement counts as a dip),
- and matches readings only if their values are within a tolerance D.

Return the maximum length of such a shared signature.

Problem Statement

You are given two integer sequences: A[1..N] from Astra-1 and B[1..M] from Astra-2, and three integers D (approximate matching tolerance), G (minimum growth per step), and K (maximum number of allowed dips). You are supposed to seek a sequence of index pairs (i_1, j_1), (i_2, j_2), $\ldots$, (i_t, j_t) such that the following constraints are met:

- **In-order constraint:** $1 \leq i_1 < i_2 < \ldots < i_t \leq N$ and $1 \leq j_1 < j_2 < \ldots < j_t \leq M$.
- **Approximate matching:** For every r in $1..t$, $|A[i_r] - B[j_r]| \leq D$.
- **Rising vs. dip steps (uses G and K):** For each consecutive pair $(i_r, j_r) \to (i_{r+1}, j_{r+1})$:
  - If $A[i_{r+1}] - A[i_r] \geq G$ and $B[j_{r+1}] - B[j_r] \geq G$, the step is considered as rising.
  - Otherwise, the step is considered as a noise dip. The total number of dip steps must be $\leq K$.

Goal: Maximize t, the length of the shared signature.

Input Format

The input will contain 3 (three) lines. First line will have space-separated 4 integers N, M, D, G, and K, where

- N, M are the lengths of the sequences,
- D is the tolerance for approximate matching $(|A[i] - B[j]| \leq D)$,
- G is the minimum required increase for a step to be considered rising,

- and K is the maximum number of allowed dip steps.

The following two lines will have the N and M space-separated integers representing the readings from Astra-1 and Astra-2 respectively. Following is the input format:

```
N M D G K
A1 A2 ... AN
B1 B2 ... BM
```

Output Format

A single integer L, the maximum length of a shared signature satisfying all rules. You need to find L using both the Recursion with Memoization and Tabulation method. See **Sample Output**s for the exact wordings.

Constraints

- $1 \leq N, M \leq 1000$
- $0 \leq D \leq 10^9$
- $0 \leq G \leq 10^9$
- $0 \leq K \leq 20$
- $|A[i]|, |B[j]| \leq 10^9$

Intended solution: $O(N \cdot M \cdot (K+1))$ **time with careful memory use.**

Rules and Clarifications

- **Matching tolerance:** You can only pair (i, j) if $|A[i] - B[j]| \leq D$.
- **Order matters:** Choose indices in strictly **increasing order** in both sequences; **skipping is allowed**.
- **Rising vs. dip:** A step is rising if both sequences increase by at least G; otherwise, it consumes one dip.
- **First element:** The first matched pair does not involve a step and does not consume a dip.
- **Edge cases:** If $K = 0$, every step must be rising. If $G = 0$, nonnegative changes count as rising; negative is a dip. If $D = 0$, matches require exact equality.
- **What to output:** Only the maximum length. Do not print the sequence itself.
- **I/O:** Read from standard input, write to standard output. Do not print extra text.

Samples

Sample Input 1

```
6 6 2 3 1
3 5 8 11 10 14
4 6 7 9 13 16
```

Sample Output 1

```
Maximum length using Recursion (with Memoization): 5
Maximum length using Tabulation: 5
```

Explanation: $D = 2, G = 3, K = 1$. One optimal chain: $(3, 4) \to (5, 6) \to (8, 9) \to (11, 13) \to (14, 16)$. The first step is a dip because the increases are only $+2/+2(< 3)$. Subsequent steps rise by at least 3 in both logs. Total length 5 with exactly one dip $\leq K$.
Note: (A_i, B_j) pair represents an approximately matched reading where A_i and B_j are from the reading lists A and B respectively.

Sample Input 2

```
5 5 1 5 0
10 15 20 18 25
9 14 19 24 30
```

Sample Output 2

```
Maximum length using Recursion (with Memoization): 4
Maximum length using Tabulation: 4
```

Explanation: $D = 1, G = 5, K = 0$. A clean rising path is $(10, 9) \to (15, 14) \to (20, 19) \to (25, 24)$, each step rising by $+5$ in both sequences. No dips are allowed or needed.

The Provided Driver File

You have been provided with a driver file called `CosmicConcordDriver.java`. This driver file is responsible for running a set of predefined test cases for the methods you will implement in `CosmicConcord.java`. A driver file is a simple program, usually a class with only a main method, whose job is to run, test, or demonstrate other classes in separate files (the ones you will create). If you open the driver file, you will notice a main method containing some code that will open the case-specific input files. Each case represents a test scenario to execute.

Please do not modify the provided driver file in any way. This includes adding or removing any content. Changes to the driver file can prevent your code from being evaluated correctly, which may result in lost points that cannot be recovered. In some cases, modifications may cause your code to fail compilation, which will have a serious impact on your grade and will not be reassessed. Graders will use a driver file with modified test cases but will rely on the same setup provided in this assignment.

Implementation Directions

You are provided with a driver file `CosmicConcordDriver.java`. You must implement the class `CosmicConcord` in `CosmicConcord.java` with the following solver method:

```
public static void solve(int[] A, int[] B, int N, int M, int D, int G, int K);
```

The solve method must call two recursive functions - *solveRecMemo* and *solveTab*, to find the solution using memoization and dynamic programming, respectively. Both results need to be printed in this 'solve' function following the same output formatting described in Sample Output. Any brute-force solution will result in a direct **zero**.
Important: Your solution must enforce all constraints: the tolerance for approximate matching, minimum required increase for a step to be considered rising, and maximum number of allowed dip steps. You must use Dynamic Programming to solve the problem. Do not write a main method. Your code will be executed using the provided driver script.

Note: You are welcome to add any other methods as long as you do not modify the driver script.

Important Guidelines:

- You **must use Dynamic Programming** to solve this problem.
- **Do not write a** `main` **function.** Your code will be executed using the provided driver script.
- Carefully review how the driver script calls your class and ensure your implementation works as expected.
- Test your code with additional test cases that you create.
- The grader will use different test cases, but they will follow the same format as the provided examples.
- Run and verify your code on **Eustis** before submitting to ensure compatibility with the grading environment.
- Follow the provided coding style guide to receive full credit.
- Add a header to your program with your name, course name, and assignment name.

Driver Contract

The driver will:

- Read N, M, D, G, K followed by N and M integers (items in A and B) respectively.
- Construct the solver as new CosmicConcord() and then call solve(A, B, N, M, D, G, K) function.
- The result needs to be printed from the solve() function, driver will print nothing.

Code Style

It is expected that students follow the code style guide requirements provided for this course. A style guide has been made available in the assignment page and students are expected to adhere to it starting with this programming assignment.

Testing my Code in Eustis

Make sure to test your code on `Eustis` before submitting your assignment, as this is the environment the TAs will use to compile and run your code. Failure to successfully compile or run on Eustis will result in a score of **zero**, even if the code runs correctly on your local machine.

Manual Run through `javac` and `java`

After moving your files into the same directory on Eustis, make sure they are compiled before testing your assignment. To compile your code, use the following command:

```
javac CosmicConcordDriver.java
```

If no messages are displayed, your code has compiled successfully with no errors or warnings. After compiling, you can execute the program by selecting the test case you wish to run. Use the following command format:

```
java CosmicConcordDriver <test_case_number>
```

For example, to run test case 2, use:

```
java CosmicConcordDriver 2
```

How graders will test your code through `python3`

Once you have completed manual testing of all test cases individually and verified that your output is correctly formatted, you may perform a final test using the Python script provided for grading purposes. This script will execute all test cases and generate a report indicating whether all tests passed or if any cases are still incorrect.

The command to run the Python script is:

```
python3 CosmicConcordTester.py
```

For more details on how this script works, please refer to the section titled *How Graders Will Test Your Code Using Python3* from the Eustis Setup assignment.

Comment Header

Please make sure to provide the appropriate comment header as the very first content at the top of your file (including before any import statements). If your comment header is incorrect or not placed literally at the top of the file, points will be deducted according to the rubric.

The header must be formatted exactly as shown below and placed on the first line of the file with no whitespace above it. For example, if your name is Steve Harrington, the comment header should look like:

```
/*  Steve Harrington
    Cosmic Concordance
    COP3503 Computer Science 2
    CosmicConcord.java
*/
```

The format is as follows:

- First line: Your first and last name.
- Second line: The assignment name.
- Third line: The course code and course name.
- Fourth line: The name of the Java file.

This format must be followed exactly for the Python script to properly capture your name. Failure to follow this format will result in point deductions. The main reason for this requirement is that graders will use a special Python script to record your name during test runs.

Grading Rubric

Please refer to Webcourses for the official grading rubric. It is located on the assignment page.

A Very Important Note for Running Eustis and Packages! Read Carefully!

Many of you are probably using IDEs like Netbeans and Eclipse to build your Java Solutions. Please note that some of these IDEs will automatically place your Java file in some sort of package. Please make sure your Java file is not defined in some package as this can result package private errors. Any such error that occurs during the grading will not be fixed and points will be deducted as such in accordance with the respective categories in the rubric. Also, DO NOT create a main method in your solution file!! This will result in your code not running properly with the runner file which will result in points being deducted from the respective categories.